


# Extra Assignment on System Design

**Name:** Antara Arifa Mullick

**Id:** 210042132

**Semester:** 5th

**Course:** Server Programming SWE 4537

**Submitted To:** Farzana Tabassum

# CineSync – Virtual Movie Theater System Design

## 1. Key Non-Functional Requirements with Justifications

1. **Scalability**

    - **Justification:** As the user base of CineSync grows, the system needs to support thousands of concurrent users. A scalable architecture, typically using a cloud-based infrastructure, ensures the application can handle a high load by adding more resources (e.g., servers or containers) dynamically. Horizontal scaling is essential to maintain performance as traffic increases.

2. **Latency and Responsiveness**

    - **Justification:** Since CineSync is a real-time synchronized movie platform, ensuring minimal latency in media streaming and user interactions is crucial. Low-latency protocols such as WebRTC (for peer-to-peer communication) or adaptive bitrate streaming are used to minimize buffering and maintain real-time synchronization, ensuring a smooth user experience.

3. **Security and Privacy**

    - **Justification:** Given the platform's interaction with sensitive data (e.g., user information, messages, shared media), strong security measures are essential. Data encryption (e.g., end-to-end encryption for chats and media), secure authentication (e.g., OAuth or JWT), and role-based access control help protect user privacy and prevent unauthorized access to content and personal data.

4. **Availability**

    - **Justification:** CineSync needs to be always available for users across different time zones and regions. Achieving high availability requires a robust system design with backup mechanisms, redundant services, and the use of Content Delivery Networks (CDNs) for faster media delivery, ensuring the platform stays online with minimal downtime.

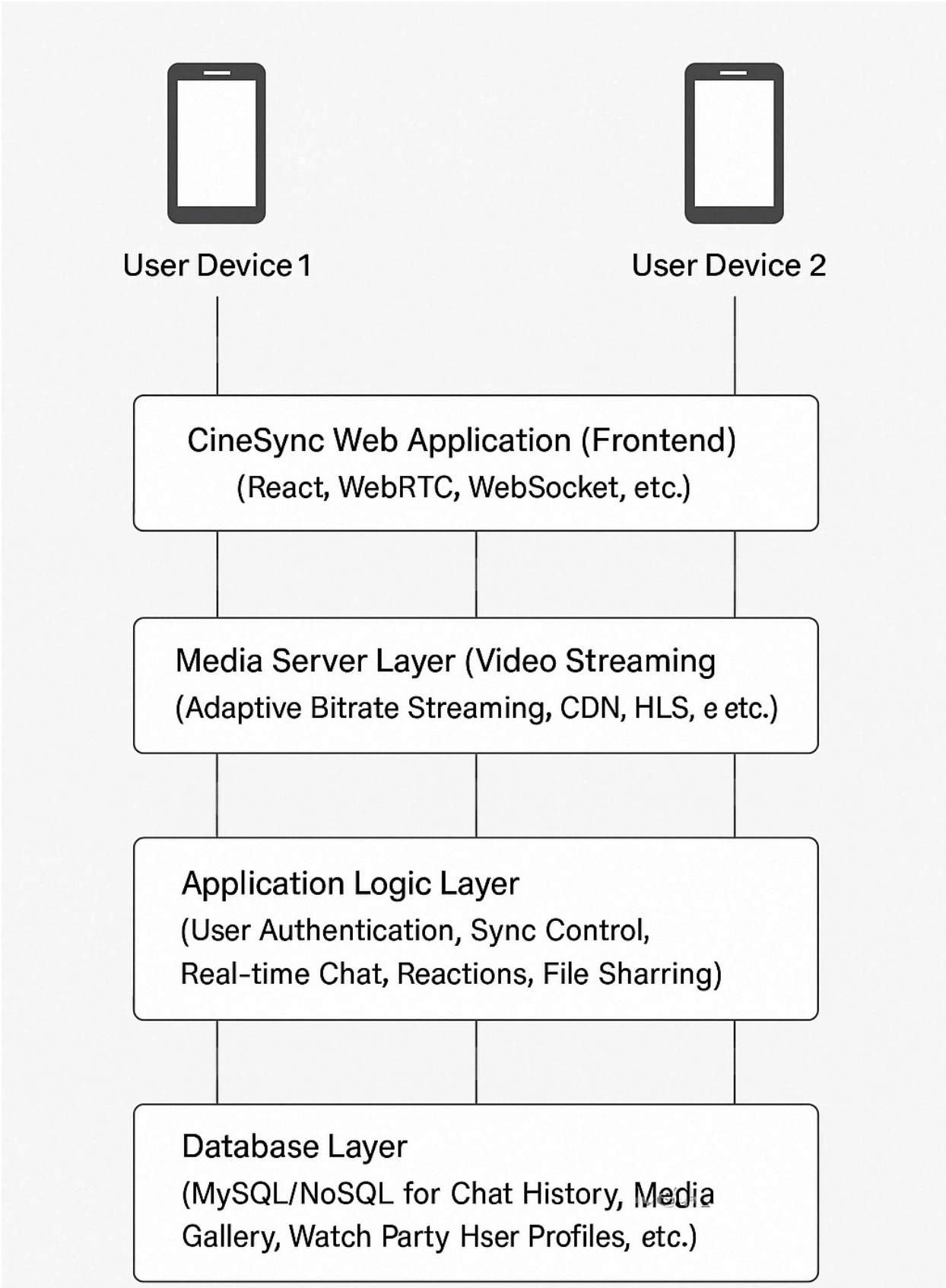
5. **Maintainability**

    - **Justification:** To ensure long-term sustainability, the system must be easy to maintain and extend. A microservices-based architecture allows teams to independently deploy and update different services (e.g., media streaming, user authentication, and chat services) without affecting the entire system. Containerization and DevOps tools like Kubernetes can streamline maintenance and upgrades.

## 2. High-Level System Design

The system design follows a microservices-based approach where different services handle specific tasks, ensuring modularity and scalability. The CineSync platform is designed to facilitate seamless interaction between users across multiple devices while maintaining real-time synchronization and media sharing.

#### System Architecture Diagram


User Device 1
User Device 2
CineSync Web Application (Frontend)
(React, WebRTC, WebSocket, etc.)
Media Server Layer (Video Streaming
(Adaptive Bitrate Streaming, CDN, HLS, e etc.)
Application Logic Layer
(User Authentication, Sync Control,
Real-time Chat, Reactions, File Sharring)
Database Layer
(MySQL/NoSQL for Chat History, Media
Gallery, Watch Party Hser Profiles, etc.)

**Description of Components:**

1. **User Device Layer (Clients)**
    - Users access the CineSync platform via their devices (mobile, tablet, or desktop). The client apps (e.g., React-based web frontend) interact with the platform to control movie playback, chat, share reactions, and upload media.
2. **CineSync Web Application (Frontend)**
    - The frontend is responsible for rendering the UI for users to interact with the platform. It includes components for movie controls (play, pause, skip), real-time chat, and reactions. It uses WebRTC for low-latency communication and WebSocket for real-time chat.
3. **Media Server Layer**
    - This layer handles the video streaming. It ensures adaptive bitrate streaming, using CDN (Content Delivery Networks) or HLS (HTTP Live Streaming) to deliver media efficiently. The server adjusts the video quality based on the user's internet connection to minimize buffering.
4. **Application Logic Layer**
    - This layer controls the logic for movie synchronization, user authentication, chat, reactions, and file-sharing. It processes user interactions (e.g., play, pause, share media) and ensures synchronization across all devices. The session store keeps track of user activities.
5. **Database Layer**
    - This layer manages databases for storing various data. MySQL or NoSQL databases store user profiles, chat history, media galleries, and watch party session data. It ensures that users' activities and shared media are preserved for future sessions.

## 3. Alternative System Design Approaches

1. **Serverless Architecture:**

    - A serverless approach could be adopted, where CineSync's backend services (such as media streaming and chat services) are deployed using serverless functions. This reduces infrastructure overhead, offering auto-scaling based on demand, while improving cost-efficiency.

2. **Peer-to-Peer (P2P) Streaming:**

    - Instead of relying on centralized servers for streaming, CineSync could explore using P2P streaming technology (such as WebRTC) for content delivery. This would reduce server load, improve scalability, and minimize latency for users with fast internet connections.

3. **Microservices and Containerization:**

    - A microservices architecture combined with containerization (using Docker and Kubernetes) could provide a flexible and maintainable solution. Each service (e.g., playback, chat, reactions) would be independent, and development teams could update and deploy services independently, ensuring continuous availability and better fault tolerance.

4. **Hybrid Streaming:**

    - Combining both live-streaming and on-demand video features could make CineSync more versatile. This hybrid model could include pre-recorded content (movies and shows) available for users to stream, while also facilitating live, real-time sync for watch parties.

## 4. Algorithms for Real-Time Synchronization

1. **Time-Synchronization Algorithm:**
    - This algorithm ensures all users in the party are watching the same part of the movie. When the host pauses, plays, or skips, the system synchronizes all other clients to the leader's timestamp. WebRTC or WebSocket technology can be used for communication between clients.
2. **Adaptive Bitrate Streaming Algorithm:**
    - This algorithm adjusts the video quality based on the user’s network conditions. It ensures that users with slower internet connections receive lower-quality video to avoid buffering, while those with faster connections get higher-quality video.
3. **Event-Driven Architecture for Chat and Reactions:**
    - The system uses event-driven architecture to manage real-time chat and emoji reactions. When a user sends a message or reaction, the event is triggered and propagated to other connected users instantly through WebSocket or XMPP.

## 5. Industry Best Practices

1. **Cloud-Native Design:**
    - Adopting cloud-native principles allows CineSync to scale effectively and stay resilient under high loads. Services such as AWS, Google Cloud, or Azure could be used for hosting the application, ensuring rapid scaling, high availability, and disaster recovery.
2. **CI/CD Pipeline:**
    - A Continuous Integration and Continuous Deployment (CI/CD) pipeline would ensure efficient testing, integration, and deployment of new features. Tools like Jenkins, GitLab, or CircleCI can be used to automate these processes, ensuring that updates are deployed without manual intervention.
3. **Focus on UX/UI:**
    - To maximize user engagement, CineSync should prioritize intuitive design. Real-time feedback (e.g., chat messages and emoji reactions) should seamlessly integrate into the movie-watching experience, and controls should be easily accessible without interrupting the content.

## 6. References


1. W3C WebRTC Specification: https://www.w3.org/TR/webrtc/
2. AWS Serverless: https://aws.amazon.com/serverless/
3. Cloud-Native Best Practices: https://www.redhat.com/en/topics/cloud-native
4. Microservices Architecture: https://www.microsoft.com/en-us/architecture/microservices